NetworkBehaviour 트레이트 구현 패턴 분석

개요

이 문서는 Rust libp2p의 `NetworkBehaviour` 트레이트 구현 패턴을 상세히 분석합니다. `NetworkBehaviour` 는 libp2p 네트워크에서 피어의 동작을 정의하는 핵심 트레이트로, 다양한 프로토콜이 이를 구현하여 네트워크 상에서의 행동을 정의합니다.

NetworkBehaviour 트레이트 정의

기본 구조

```rust
pub trait NetworkBehaviour: 'static {
/// Handler for all the protocols the network behaviour supports.
type ConnectionHandler: ConnectionHandler;

/// Event generated by the `NetworkBehaviour` and that the swarm will report bac
type ToSwarm: Send + 'static;

// 연결 생명주기 관리 메서드들
fn handle_pending_inbound_connection(&mut self, ...) -> Result<(), ConnectionDe
fn handle_established_inbound_connection(&mut self, ...) -> Result<THandler<Sel
fn handle_pending_outbound_connection(&mut self, ...) -> Result<Vec<Multiaddr>,
fn handle_established_outbound_connection(&mut self, ...) -> Result<THandler<Se
```

```
// 이벤트 처리 메서드들
fn on_swarm_event(&mut self, event: FromSwarm);
fn on_connection_handler_event(&mut self, peer_id: PeerId, connection_id: Conne

// 폴링 메서드
fn poll(&mut self, cx: &mut Context<'_>) -> Poll<ToSwarm<Self::ToSwarm, THandle
}
```

핵심 연관 타입 (Associated Types)

1. **ConnectionHandler**: 각 연결에 대한 프로토콜 처리를 담당
2. **ToSwarm**: Swarm에게 전달할 이벤트 타입

구현 패턴 분석

1. 단순한 프로토콜 패턴 (Ping)

특징:

- 최소한의 상태 관리
- 단순한 이벤트 큐 기반 구조
- 연결별 동일한 핸들러 생성

```
pub struct Behaviour {
config: Config,
```

```
events: VecDeque<Event>,
}

impl NetworkBehaviour for Behaviour {
type ConnectionHandler = Handler;
type ToSwarm = Event;

fn handle_established_inbound_connection(&mut self, ...) -> Result<THandler<Self
Ok(Handler::new(self.config.clone()))
}

fn handle_established_outbound_connection(&mut self, ...) -> Result<THandler<Sel
Ok(Handler::new(self.config.clone()))
}

fn on_connection_handler_event(&mut self, peer: PeerId, connection: ConnectionId
self.events.push_front(Event { peer, connection, result })
}

fn poll(&mut self, _: &mut Context<'_>) -> Poll<ToSwarm<Self::ToSwarm, THandlerI
if let Some(e) = self.events.pop_back() {
Poll::Ready(ToSwarm::GenerateEvent(e))
} else {
```

```
Poll::Pending
}
}

fn on_swarm_event(&mut self, _event: FromSwarm) {}
}
```

패턴 특징:

- **이벤트 큐**: `VecDeque<Event>` 로 발생한 이벤트를 순차 처리
- **상태 무관성**: 연결 상태에 관계없이 동일한 핸들러 생성
- **단순한 폴링**: 큐에서 이벤트를 꺼내어 반환하는 단순한 구조

2. 상태 관리 패턴 (Identify)

특징:

- 연결별 상태 추적
- 주소 관리 및 변경 감지
- 복잡한 이벤트 처리 로직

```
pub struct Behaviour {
config: Config,
connected: HashMap<PeerId, HashMap<ConnectionId, Multiaddr>>,
our_observed_addresses: HashMap<ConnectionId, Multiaddr>,
outbound_connections_with_ephemeral_port: HashSet<ConnectionId>,
```

```
events: VecDeque<ToSwarm<Event, InEvent>>,

discovered_peers: PeerCache,

listen_addresses: ListenAddresses,

external_addresses: ExternalAddresses,

}
```

핵심 패턴:

- **연결 상태 추적**: `HashMap<PeerId, HashMap<ConnectionId, Multiaddr>>` 로 피어별 연결 관리
- **주소 관리**: 리슨 주소와 외부 주소 변경 감지 및 처리
- **캐시 시스템**: 발견된 피어 정보를 LRU 캐시로 관리

```
fn on_swarm_event(&mut self, event: FromSwarm) {

let listen_addr_changed = self.listen_addresses.on_swarm_event(&event);

let external_addr_changed = self.external_addresses.on_swarm_event(&event);



if listen_addr_changed || external_addr_changed {

// 모든 연결된 핸들러에게 주소 변경 알림

let change_events = self.connected.iter()

.flat_map(|(peer, map)| map.keys().map(|id| (*peer, id)))

.map(|(peer_id, connection_id)| ToSwarm::NotifyHandler {

peer_id,

handler: NotifyHandler::One(*connection_id),

event: InEvent::AddressesChanged(self.all_addresses()),
```

```
})
.collect::<Vec<_>>();


self.events.extend(change_events)
}
}
```

3. 복잡한 쿼리 기반 패턴 (Kademlia)

특징:

- 라우팅 테이블 관리
- 복잡한 쿼리 시스템
- 다중 작업 스케줄링

```
pub struct Behaviour<TStore> {
kbuckets: KBucketsTable<kbucket::Key<PeerId>, Addresses>,
queries: QueryPool,
connected_peers: FnvHashSet<PeerId>,
add_provider_job: Option<AddProviderJob>,
put_record_job: Option<PutRecordJob>,
queued_events: VecDeque<ToSwarm<Event, HandlerIn>>,
store: TStore,
// ... 기타 상태 필드들
}
```

핵심 패턴:

- **라우팅 테이블**: K-bucket 기반 피어 관리
- **쿼리 풀**: 동시 다발적 쿼리 관리
- **작업 스케줄링**: 주기적 작업 (재발행, 복제 등) 관리

```
fn handle_pending_outbound_connection(
&mut self,
_connection_id: ConnectionId,
maybe_peer: Option<PeerId>,
_addresses: &[Multiaddr],
_effective_role: Endpoint,
) -> Result<Vec<Multiaddr>, ConnectionDenied> {
let Some(peer_id) = maybe_peer else {
return Ok(vec![]);
};


// K-bucket에서 피어 주소 조회
let key = kbucket::Key::from(peer_id);
let mut peer_addrs = if let Some(kbucket::Entry::Present(mut entry, _)) = self.k
entry.value().iter().cloned().collect::<Vec<_>>()
} else {
Vec::new()
};
```

```
// 진행 중인 쿼리에서 추가 주소 수집
for query in self.queries.iter() {
if let Some(addrs) = query.peers.addresses.get(&peer_id) {
peer_addrs.extend(addrs.iter().cloned())
}
}

Ok(peer_addrs)
}
```

공통 구현 패턴

1. 이벤트 큐 패턴

모든 구현체에서 공통적으로 사용:

```
events: VecDeque<ToSwarm<Event, HandlerInEvent>>

fn poll(&mut self, _: &mut Context<'_>) -> Poll<ToSwarm<Self::ToSwarm, THandlerI
if let Some(event) = self.events.pop_front() {
return Poll::Ready(event);
}
Poll::Pending
```

```
}
```

특징:

- FIFO 방식으로 이벤트 처리
- 비동기 처리를 위한 이벤트 버퍼링
- 백프레셔 방지를 위한 큐 크기 제한 (일부 구현체)

2. 연결 생명주기 관리 패턴

연결 설정 시:

```
fn handle_established_inbound_connection(&mut self, ...) -> Result<THandler<Self
// 1. 연결 정보 저장
self.connected.entry(peer_id).or_default().insert(connection_id, addr);
// 2. 핸들러 생성 및 초기화
let mut handler = Handler::new(config, endpoint, peer_id);
self.preload_handler(&mut handler);
Ok(handler)
}
```

연결 종료 시:

```
fn on_swarm_event(&mut self, event: FromSwarm) {
match event {
FromSwarm::ConnectionClosed(ConnectionClosed { peer_id, connection_id, remaining
```

```
if remaining_established == 0 {
self.connected.remove(&peer_id);
} else if let Some(addrs) = self.connected.get_mut(&peer_id) {
addrs.remove(&connection_id);
}
}
// ...
}
}
```

3. 핸들러 통신 패턴

핸들러로 이벤트 전송:

```
self.events.push_back(ToSwarm::NotifyHandler {
peer_id,
handler: NotifyHandler::One(connection_id), // 또는 NotifyHandler::Any
event: handler_event,
});
```

핸들러로부터 이벤트 수신:

```
fn on_connection_handler_event(&mut self, peer_id: PeerId, connection_id: Connec
match event {
HandlerEvent::RequestReceived { request, request_id } => {
```

```
// 요청 처리 로직
self.handle_request(peer_id, connection_id, request, request_id);
}
HandlerEvent::ResponseReceived { response, query_id } => {
// 응답 처리 로직
self.handle_response(peer_id, response, query_id);
}
// ...
}
}
```

고급 패턴

1. 주소 관리 패턴

외부 주소 후보 보고:

```
// 새로운 외부 주소 후보 발견 시
self.events.push_back(ToSwarm::NewExternalAddrCandidate(observed_addr));

// 외부 주소 확인 시
self.events.push_back(ToSwarm::ExternalAddrConfirmed(confirmed_addr));
```

주소 변환 처리:

```
if self.outbound_connections_with_ephemeral_port.contains(&connection_id) {
// 임시 포트를 실제 리슨 포트로 변환
let translated_addresses = self.listen_addresses.iter()
.filter_map(|server| _address_translation(server, observed))
.collect();
}
```

2. 쿼리 관리 패턴

쿼리 시작:

```
pub fn get_record(&mut self, key: &record::Key) -> QueryId {
let query_id = QueryId::new();
let query = Query::new(QueryInfo::GetRecord { key: key.clone() });
self.queries.add_query(query_id, query);
// 초기 피어들에게 쿼리 전송
self.start_query_with_peers(query_id, initial_peers);
query_id
}
```

쿼리 진행:

```
fn discovered(&mut self, query_id: &QueryId, source: &PeerId, peers: impl Iterat
if let Some(query) = self.queries.get_mut(query_id) {
```

```
for peer in peers {
query.add_peer(peer.node_id, peer.multiaddrs.clone());
}
// 다음 단계 피어들에게 쿼리 계속
self.continue_query(query_id);
}
}
```

3. 주기적 작업 패턴

작업 스케줄링:

```
struct PeriodicJob {
interval: Duration,
next_run: Instant,
}

impl PeriodicJob {
fn poll(&mut self, now: Instant) -> bool {
if now >= self.next_run {
self.next_run = now + self.interval;
true
} else {
false
```

```
}
}
}
```

폴링에서 작업 실행:

```
fn poll(&mut self, cx: &mut Context<'_>) -> Poll<ToSwarm<Self::ToSwarm, THandler
let now = Instant::now();
// 주기적 작업 확인
if let Some(ref mut job) = self.periodic_job {
if job.poll(now) {
self.execute_periodic_task();
}
}
// 이벤트 처리
if let Some(event) = self.events.pop_front() {
return Poll::Ready(event);
}
Poll::Pending
}
```

모범 사례 (Best Practices)

1. 상태 관리

- **연결 상태 추적**: 피어별, 연결별 상태를 명확히 관리
- **리소스 정리**: 연결 종료 시 관련 상태 정리
- **메모리 제한**: 캐시나 큐 크기 제한으로 메모리 사용량 제어

2. 이벤트 처리

- **이벤트 순서**: FIFO 큐로 이벤트 순서 보장
- **백프레셔**: 큐 크기 제한으로 메모리 오버플로우 방지
- **에러 처리**: 핸들러 에러를 적절히 처리하고 복구

3. 성능 최적화

- **배치 처리**: 여러 이벤트를 한 번에 처리
- **지연 초기화**: 필요할 때만 리소스 할당
- **효율적인 자료구조**: HashMap, HashSet 등 적절한 자료구조 사용

4. 확장성

- **모듈화**: 기능별로 모듈 분리
- **설정 가능**: Config 구조체로 동작 커스터마이징
- **플러그인 아키텍처**: 다른 Behaviour와 조합 가능한 설계

결론

NetworkBehaviour 트레이트는 libp2p에서 프로토콜 동작을 정의하는 핵심 인터페이스입니다. 구현 복잡도에 따라 단순한 이벤트 큐 패턴부터 복잡한 상태 관리 및 쿼리 시스템까지 다양한 패턴이 사용됩니다. 성공적인 구현을 위해서는 연결 생명주기 관리, 이벤트 처리, 상태 관리 등의 핵심 패턴을 이해하고 적절히 적용하는 것이 중요합니다.